Atcoder Educational DP Contest 题解

张晴川

March 28, 2020

Contents

A Frog 1 **4**

B Frog 2 **5**

C Vacation **6**

D Knapsack 1 **7**

E Knapsack 2 **8**

F LCS **9**

G Longest Path **10**

H Grid 1 **11**

I Coins **12**

J Sushi **13**

K Stones **16**

L Deque **17**

M Candies **18**

N Slimes **19**

O Matching **21**

P Independent Set **23**

Q Flowers **24**

R **26**

S **27**

T **28**

U **29**

V **30**

W **31**

X **32**

Y Grid 2 **33**

Z Frog 3 **36**

前言

本文为 **Atcoder** 上的 *Educational DP Contest* 的简要题解。该场比赛共有 26 题，从易到难，是练习动态规划的优秀之选。

提交地址为: `https://atcoder.jp/contests/dp`

本文会在我的github上持续更新。如发现错误或有修改建议，请在 Github 上提 issue。如觉得有帮助，不妨 star 以支持我继续写作。

A Frog 1

题意

有 n 块石头排成一排，第 i 块石头的高度为 h_i。青蛙现在站在第 1 块石头上。它可以从第 i 块石头跳到第 $i+1$ 或者 $i+2$ 块石头上。从第 i 块石头跳到第 j 块需要消耗 $|h_j - h_i|$ 点体力。请求出跳到第 n 块石头所需的最小体力值。

数据范围

- $1 \le n \le 10^5$

题解

设 `dp[i]` 为跳到第 i 块石头上所需的最小体力值，则 `dp[n]` 为题之所求。根据题意，不难发现有：

$$\texttt{dp[i]} = \begin{cases} 0 & i = 1 \\ |h_2 - h_1| & i = 2 \\ \min(\texttt{dp[i-1]} + |h_i - h_{i-1}|, \texttt{dp[i-2]} + |h_i - h_{i-2}|) & i \ge 3 \end{cases}$$

$O(N)$ 递推一下即可。

核心代码

```
dp[1] = 0
dp[2] = abs(h[2]-h[1]);
for(int i = 2;i<=n;i++){
    dp[i] = min(dp[i-1]+abs(h[i]-h[i-1]),dp[i-2]+abs(h[i]-h[i-2]));
}
```

B Frog 2

题意

有 n 块石头排成一排，第 i 块石头的高度为 h_i。青蛙现在站在第 1 块石头上。它可以从第 i 块石头跳到第 $i+1, i+2, \ldots, i+k$ 块石头上。从第 i 块石头跳到第 j 块需要消耗 $|h_j - h_i|$ 点体力。请求出跳到第 n 块石头所需的最小体力值。

数据范围

- $1 \le n \le 10^5$
- $1 \le k \le 100$

题解

设 `dp[i]` 为跳到第 i 块石头上所需的最小体力值，则 `dp[n]` 为题之所求。根据题意，不难发现有：

$$\texttt{dp[i]} = \begin{cases} 0 & i = 1 \\ \min\{\texttt{dp[j]} + |h_i - h_j| \mid i - k \le j < i\} & i \ge 2 \end{cases}$$

$O(nk)$ 递推一下即可。

核心代码

```
dp[1] = 0
for(int i = 2;i<=n;i++){
    for(int j = max(i-k,1);j<i;j++){
        dp[i] = min(dp[i],dp[j]+abs(h[i]-h[j]));
    }
}
```

C Vacation

题意

太郎的暑假有 n 天，第 i 天他可以选择以下三种活动之一：

- 游泳，获得 a_i 点幸福值。
- 捉虫，获得 b_i 点幸福值。
- 写作业，获得 c_i 点幸福值。

但他不能连续两天进行同一种活动，请求出最多可以获得多少幸福值。

数据范围

- $1 \le n \le 10^5$

题解

设 A[i],B[i],C[i] 分别表示在第 i 天进行三种活动的前提下，前 i 天的最大幸福值。那么可以得到，

$$\texttt{A[i]} = \begin{cases} a_1 & i = 1 \\ \max(\texttt{B[i-1]}, \texttt{C[i-1]}) + a_i & i \ge 2 \end{cases}$$

B[i] 和 C[i] 的求法类似，故不赘述。实现时可以开一个大小为 $n \times 3$ 的二维数组。

核心代码

```
//v[d][i] 表示第 d 天进行第 i 项活动获得的幸福值
for(int i = 0;i<3;i++){
    dp[1][i] = v[1][i];
}
for(int day = 2;day<=n;day++){
    for(int cur = 0;cur<3;cur++){
        dp[day][cur] = 0;
        for(int last = 0;last<3;last++){
            if(cur != last){
                dp[day][cur] = max(dp[day][cur],dp[day-1][last] + v[day][cur]);
            }
        }
    }
}
```

D Knapsack 1

题意

经典 01 背包问题。有 n 个物品，每个物品有重量 w_i 和价值 v_i。背包的容量为 W，换句话说，可以放进背包的物品的重量总和不能超过 W。请最大化被选取物品的价值总和。

数据范围

- $1 \le n \le 100$
- $1 \le W \le 10^5$
- $1 \le v_i \le 10^9$

题解

我们设 `dp[i][j]` 表示 **"只考虑前 i 个物品的情况下，背包容量是 j 所能凑出的最大价值之和"**。那么在计算 `dp[i][j]` 的时候，所有情况可以分成两类考虑，

1. **拿第 i 件物品**，那么现在背包容量只剩下 $j - w_i$，而由于每样物品只能拿一件，所以我们只需要考虑前 $i-1$ 件物品的最优选取方式，即最终价值为 $\texttt{dp}[i-1][j-w_i] + v_i$。
2. **不拿第 i 件物品**，那么背包容量仍然是 j。由于我们选择不拿第 i 件物品，现在只需要考虑前 $i-1$ 件物品的最优选取方式，即最终价值为 $\texttt{dp}[i-1][j]$。

所以可以得到转移方程为：

$$\texttt{dp[i][j]} = \begin{cases} 0 & i = 0 \\ \max(\texttt{dp}[i-1][j-w_i] + v_i, \texttt{dp}[i-1][j]) & i \ge 1 \end{cases}$$

递推或记忆化搜索的复杂度均为 $O(nW)$。

核心代码

```cpp
//注意要开 long long
for(int i = 1;i<=n;i++)
    for(int j = 0;j<=W;j++)
        if(j < w[i])
        //容量不够
            dp[i][j] = dp[i-1][j];
        else
        //容量足够
            dp[i][j] = max(dp[i-1][j-w[i]] + v[i], dp[i-1][j]);
```

E Knapsack 2

题意

同上题，仅数据范围不同：

数据范围

- $1 \le n \le 100$
- $1 \le W \le \mathbf{10^9}$
- $1 \le v_i \le \mathbf{10^3}$

题解

由于 W 达到了 10^9 的量级，之前的 $O(nW)$ 算法无法通过，但由于每样物品的价值上限仅为 10^3，我们可以另辟蹊径。设 `dp[i][j]` 表示 **"只考虑前 i 个物品的情况下，总价值是 j 所需的最小容量"**。那么在计算 `dp[i][j]` 的时候，所有情况依然可以分成两类考虑：

1. **拿第 i 件物品**，那么别的物品的总价值需要凑出 $j - v_i$，而由于每样物品只能拿一件，所以我们只需要考虑前 $i-1$ 件物品的最优选取方式，即最终重量为 $\texttt{dp}[i-1][j-v_i] + w_i$。
2. **不拿第 i 件物品**，那么别的物品需要凑出 j 的价值。由于我们选择不拿第 i 件物品，现在只需要考虑前 $i-1$ 件物品的最优选取方式，即最小重量为 $\texttt{dp}[i-1][j]$。

计算完所有状态的值后，只需要选取满足重量上限的最大价值即可。

核心代码

```
int bound = n*1000;//最大价值
for(int v = 0;v<=bound;v++)
    if(v == 0) dp[0][v] = 0;
    else dp[0][v] = inf;
for(int i = 1;i<=n;i++)
    for(int tot = 0;tot<=bound;tot++)
        if(tot < v[i])
            dp[i][tot] = dp[i-1][tot]
        else
            dp[i][tot] = min(dp[i-1][tot],dp[i-1][tot-v[i]] + w[i]);
answer = 0
for(int tot = 0;tot<=bound;tot++)
    if(dp[n][tot] <= W)
        answer = tot
```

F LCS

题意

给定字符串 s 和 t，求最长公共子序列（不需要连续）。

数据范围

- $1 \le |s|, |t| \le 3000$

题解

设 s, t 的长度分别为 n, m。现在考虑两个串的最后字符，一共有两种情况：

1. $s_n = t_m$，那么答案最末字符显然等于也等于它们俩。
2. $s_n \ne t_m$, 那么两者必有其一不在答案中，所以答案为 $\texttt{LCS}(s_{1\ldots n}, t_{1\ldots m-1})$ 和 $\texttt{LCS}(s_{1\ldots n-1}, t_{1\ldots m})$ 的较长者。

用 `dp[i][j]` 表示 s 的前 i 个字符和 t 的前 j 个字符的 `LCS` 长度。那么可以得到转移方程

$$\texttt{dp[i][j]} = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ \texttt{dp[i-1][j-1]} + 1 & \texttt{s[i]} == \texttt{t[j]} \\ \max(\texttt{dp[i-1][j]}, \texttt{dp[i][j-1]}) & \text{otherwise} \end{cases}$$

答案本身只需要通过 `dp[i][j]` 的转移来源恢复即可，参考以下代码。

核心代码

```cpp
// 计算最优长度
for(int i = 1; i <= n; i++)
    for(int j = 1; j <= m; j++)
        if(s[i] == t[j])
            dp[i][j] = dp[i - 1][j - 1] + 1;
        else
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
// 恢复答案
int i = n, j = m;
while(dp[i][j] > 0)
    if(s[i] == t[j]) {
        ans[dp[i][j]] = s[i];
        i--;
        j--;
    } else {
        if(dp[i][j] == dp[i - 1][j])i--;
        else j--;
    }
```

G Longest Path

题意

给定一个 n 个点，m 条边的**有向无环图**，求图中最长路径的长度。定义路径长度为边的数量。

数据范围

- $2 \le N \le 10^5$
- $1 \le M \le 10^5$

题解

本题是**所有**动态规划模型的**理论**基础。抽象地说，所有动态规划的本质都是在一张（带权）有向无环图上求最长路，之前做的背包，最长公共子序列皆是如此。对应关系如下：

动态规划	有向无环图
状态	节点
转移方程	边
无后效性	图中无环
···	···

这题的转移方程比较显然，即 dp[i] 表示以节点 i 为终点的路径中最长的长度。转移方程为：

$$\texttt{dp[i]} = \max\{\texttt{dp[j]} + 1 \mid \text{存在边 } j \to i\}$$

有向无环图最重要的性质就是没有环，这使得我们可以对节点进行拓扑排序，并按照拓扑序的顺序计算 dp 状态的值。相信你对于宽度优先搜索（BFS）没有什么问题，以下我给出一种用深度优先搜索（DFS）的做法，又称记忆化搜索。

核心代码

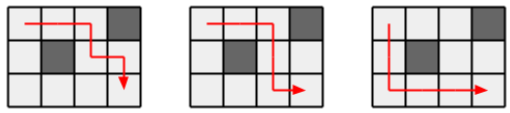
```
void calc(int i) {
    if(vis[i])
        return dp[i];
    else {
        dp[i] = 0;
        vis[i] = true;
        for(auto j:reversed_edges[i]) {
            dp[i] = max(dp[i], calc(j) + 1);
        }
        return dp[i];
    }
}
```

H　Grid 1

题意

给定一个 h 行 w 列的网格，现在位于 $(1,1)$，每次可以向下或右走一格，求有多少种方式（对 10^9+7 取模）走到 (h,w)。

网格中有若干个不能经过的障碍，但不会出现在起点和终点。



数据范围

- $2 \le h, w \le 1000$

题解

这题比较简单，用 `dp[i][j]` 表示走到 (i,j) 的方案数，只需要枚举来源即可。转移方程为：

$$\texttt{dp[i][j]} = \begin{cases} 0 & (i,j) \text{ 是障碍} \\ \texttt{dp[i-1][j]} + \texttt{dp[i][j-1]} & \text{otherwise} \end{cases}$$

顺带一提，如果没有障碍，这个 dp 表格就是 杨辉三角，即 $\texttt{dp[i][j]} = \binom{i+j}{i}$ 。

核心代码

```
dp[1][1] = 1;
for(int i = 1; i <= h; i++) {
    for(int j = 1; j <= w; j++) {
        if(i == 1 and j == 1)continue;
        if(dp[i][j] == '.') //如果可以经过
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
    }
}
```

I Coins

题意

有 n 枚硬币排成一排。现在同时抛所有硬币，第 i 枚硬币向上的概率是 p_i，向下的概率是 $1-p_i$，求向上的硬币数量比向下的多的概率。

数据范围

- $1 \le n \le 2999$
- n 是奇数

题解

这题依旧很简单，设 `dp[i][j]` 表示前 i 个元素有 j 个向上的概率。只需要枚举当前硬币是向上还是向下即可，与背包类似。转移方程为：

$$\texttt{dp[i][j]} = \texttt{dp[i-1][j-1]} \times p_i + \texttt{dp[i-1][j]} \times (1-p_i)$$

计算完毕后，枚举有多少枚硬币向上就做完了。

核心代码

```
prob[0][0] = 1;
for(int i = 1;i<=n;i++){
    dp[i][0] = dp[i-1][0] * (1-p[i]);
    for(int j = 1;j<=i;j++){
        dp[i][j] = dp[i-1][j-1] * p[i] + dp[i-1][j] * (1-p[i]);
    }
}

ans = 0;
for(int j = 0;j<=n;j++){
    if(j > n-j){
        ans += dp[n][j];
    }
}
```

J Sushi

题意

现在有 n 盘寿司在桌上排成一排，第 i 盘寿司里面有 a_i 个寿司。直到吃完所有寿司为止，你都会持续进行下述操作：

- 从 $\{1, 2, \cdots, n\}$ 中**均匀随机**选取一个数 i。如果第 i 盘中仍有寿司，那就吃一块，否则什么也不做。

现在求期望操作次数是多少。

数据范围

- $1 \le n \le 300$
- $1 \le a_i \le 3$

题解

暴力 DP

首先我们考虑最暴力的做法，用 $\texttt{dp}[c_1][c_2]\cdots[c_n]$ 表示第 i 盘还剩 c_i 个寿司的期望次数。那么枚举随机到的数 i 就可以得到方程：

$$\texttt{dp}[c_1][c_2]\cdots[c_n] = 1 + \sum_{i=1}^{n} \frac{1}{n}\texttt{dp}[c_1][c_2]\cdots[\max(c_i - 1, 0)]\cdots[c_n]$$

合并状态

（注意该方程并不能构成**转移**方程，因为当第 i 盘已经为空的时候，状态不变）

我们现在考虑**合并等价状态**（题外话：动态规划优化的初等方法无非就那么几种，合并状态就是最重要的思想之一）。

注意到由于随机数是均匀选取的，那么盘子的位置是无关紧要的，而只有剩余数量有影响。于是相同数值的不同排列的期望次数必然是相同的。例如 $\texttt{dp}[1][2][3] = \texttt{dp}[3][2][1]$。

所以只需要考虑每种数值出现的次数。因为 $a_i \le 3$，所以至多有四种不同数值：$\{0, 1, 2, 3\}$。我们重新定义状态 `dp[a][b][c][d]` 表示当前还剩下 $a/b/c/d$ 盘有 $0/1/2/3$ 个寿司。

有了状态，我们通过枚举当前随机到的盘子里还剩几只寿司得到如下方程：

$$\begin{aligned}\texttt{dp[a][b][c][d]} = 1 &+ \frac{a}{n}\texttt{dp[a][b][c][d]} \\ &+ \frac{b}{n}\texttt{dp[a+1][b-1][c][d]} \\ &+ \frac{c}{n}\texttt{dp[a][b+1][c-1][d]} \\ &+ \frac{d}{n}\texttt{dp[a][b][c+1][d-1]}\end{aligned}$$

移项整理得到**转移**方程：

$$\begin{aligned}\texttt{dp[a][b][c][d]} = &\frac{n}{b+c+d} \\ &+ \frac{b}{b+c+d}\texttt{dp[a+1][b-1][c][d]} \\ &+ \frac{c}{b+c+d}\texttt{dp[a][b+1][c-1][d]} \\ &+ \frac{d}{b+c+d}\texttt{dp[a][b][c+1][d-1]}\end{aligned}$$

消除无用状态

但由于 $n \le 300$，总状态数高达 $300^4 \approx 8 \times 10^9$ 无法通过。我们需要进一步优化。

注意到任意时刻下，$a+b+c+d$ 的值都应该等于 n：因为不管进行多少次操作，盘子总数总是 n。所以其实只需要知道 b, c, d 就可以反推出 a 的值：$a = n - (b+c+d)$。那么现在只保留后面三维，得到转移方程：

$$\begin{aligned}\texttt{dp[b][c][d]} = &\frac{n}{b+c+d} \\ &+ \frac{b}{b+c+d}\texttt{dp[b-1][c][d]} \\ &+ \frac{c}{b+c+d}\texttt{dp[b+1][c-1][d]} \\ &+ \frac{d}{b+c+d}\texttt{dp[b][c+1][d-1]}\end{aligned}$$

至此，足矣。

核心代码

```
double calc(int b, int c, int d) {
    if(min({b,c,d}) < 0)return 0;
    if(vis[b][c][d])
        return dp[b][c][d];
    else {
        vis[b][c][d] = true;
        double sum = b + c + d;
        dp[b][c][d] = n / sum
                    + b / sum * calc(b - 1, c, d)
                    + c / sum * calc(b + 1, c - 1, d)
                    + d / sum * calc(b, c + 1, d - 1);
        return dp[b][c][d];
    }
}
```

K Stones

题意

给定一个有 n 个正整数的集合 $A = \{a_1, a_2, \ldots, a_n\}$。

现在有 k 块石头和两名玩家。两人轮流进行以下操作，不能进行操作者输：

- 从 A 中指定一个正整数 a_i，并移走恰好 a_i 块石头。

注：每个 a_i 可以重复使用。

数据范围

- $1 \le n \le 100$
- $1 \le k \le 10^5$
- $1 \le a_1 < a_2 < \cdots < a_n \le k$

题解

本题涉及到的知识点是**博弈论**。

注意到以下事实：

1. 剩余 0 块石头时，当前操作者必败。
2. 如果可以使得对手必败，那么当前操作者必胜。
3. 反之，如果不论进行何种操作对手都将必胜，那么当前玩家只能认输。

那么我们设 `dp[i]` 表示还剩 i 块石头时，当前操作者是否必胜，那么 `dp[i]` 为**必胜**当且仅当存在 $i \ge a_j$ 且 $\texttt{dp}[i - a_j]$ 为必败。

核心代码

```
dp[0] = false;//终局必败
for(int i = 1;i<=k;i++){
    //默认必败
    dp[i] = false;
    for(int j = 0;j<n;j++){
        //如果存在方法使得对手必败，那么当前必胜
        if(i >= a_j and dp[i-a_j] == false){
            dp[i] = true;
        }
    }
}
```

L Deque

题意

给定一个双端队列 $a = (a_1, a_2, \ldots, a_n)$。现在有两名玩家轮流进行以下操作，直至队列为空：

- 从队首或队尾取走一个数 c，并且本次收益为 c。

设先/后手的总收益分别为 X 和 Y。先手想要最大化 $X - Y$，后手想要最大化 $Y - X$，即两人都想最大化自己与对手的差值。现两名玩家均以**最优策略**操作，请问最终 $X - Y$ 的值是多少？

数据范围

- $1 \le n \le 3000$
- $1 \le a_i \le 10^9$

题解

其实本题核心就在 **"最优策略"** 这四个字的定义。什么是最优策略？就是选取最坏情况下收益最高的决策。什么是最坏情况呢？这里当然不是指你脑子短路故意选取一个非最优解，而是指假设对方也会以最优策略决策。细心的同学会发现这里的定义是递归的，但这并没有关系，因为在只剩下一个数，或者更平凡的——序列为空的情况下，只存在一种决策供选择。

设 `dp[i][j]` 表示序列为 $a_i, a_{i+1}, \ldots, a_j$ 时，先手总收益减后手总收益的最大值。那么我们可以得到转移方程：

$$\texttt{dp[i][j]} = \max(a_i - \texttt{dp[i+1][j]}, a_j - \texttt{dp[i][j-1]})$$

注意到上式中是减号的原因是，当前的先手在拿完后就变为下一轮的后手，所以当前先手的收益为下一轮中后手的收益。可能比较绕，多思考一下即可。

核心代码

```
void solve(int l,int r){
    if(dp[l][r] != -1)return;
    else{
        if(l == r){
            dp[l][r] = a[l];
        }else{
            solve(l+1, r);
            solve(l, r-1);
            dp[l][r] = max(a[l]-dp[l+1][r],a[r]-dp[l][r-1]);
        }
    }
}
```

M Candies

题意

现在有 n 个小孩和 k 颗糖。第 i 个小孩获得的糖数必须在 $[0, a_i]$ 之内，求一共有多少种把糖发完的方法。

数据范围

- $1 \le n \le 100$
- $0 \le k \le 10^5$
- $0 \le a_i \le k$

题解

有了之前背包的经验，显然我们可以设状态 `dp[i][j]` 表示前 i 个人一共发了 j 颗糖的方法，并且转移方程就是枚举当前的人获得的糖数 c：

$$\texttt{dp[i][j]} = \sum_{c=0}^{a_i} \texttt{dp[i-1][j-c]}$$

但可达鸭眉头一皱，发现事情并不简单。注意到状态总数为 10^7，而每个状态最多有 $k \le 10^5$ 个转移，总复杂度高达 10^{12}，于是无法通过。

我们仔细观察上面的转移方程，可以发现同一行相邻的两个状态其实差别不大：

$$\texttt{dp[i][j]} = \texttt{dp[i-1][j-c]} + \cdots + \texttt{dp[i-1][j]}$$

$$\texttt{dp[i][j+1]} = \texttt{dp[i-1][j-c+1]} + \cdots + \texttt{dp[i-1][j+1]}$$

通过比对，可以看到其实只首尾两项，于是得到：

$$\texttt{dp[i][j+1]} = \texttt{dp[i][j]} - \texttt{dp[i-1][j-c]} + \texttt{dp[i-1][j+1]}$$

现在计算每个状态只需要 $O(1)$ 次运算，可以通过。

核心代码

```
dp[0][0] = 1;
for(int i = 1; i <= n; i++) {
    dp[i][0] = 1;
    for(int j = 0; j < k; j++) {
        dp[i][j + 1] = dp[i][j] + dp[i - 1][j + 1];
        if(j - a[i] >= 0)
            dp[i][j + 1] -= dp[i - 1][j - a[i]];
    }
}
```

注：本题也可以使用前缀和加速，原理大同小异。

N Slimes

题意

给定序列 $a = [a_1, a_2, \cdots, a_n]$。

每次操作可以把**相邻**的两个元素合并，新元素的值为权值之和。每次合并的成本为新元素的值。

求把所有元素合并成一个的最小成本。

数据范围

- $2 \le n \le 400$
- $1 \le a_i \le 10^9$

题解

本题是一道典型的**区间型**动态规划。

注意到下列事实：

1. 无论顺序为何，若干元素合并后的权值一定等于初始权值之和。
2. 由于只能合并相邻元素，在**任意**时刻，序列中的每个元素一定对应初始序列的某个连续段：例如 $[1, 2, 3, 4, 5]$ 经过若干次操作后变为 $[(1{+}2{+}3), (4{+}5)]$，则 $(1 + 2 + 3)$ 对应 $[1, 2, 3]$ 。

现在我们来观察将一个序列合并成单独一个元素的过程：$[a_1, a_2, \cdots, a_n] \to s$。不难发现一定是头部若干元素合并后的结果与尾部若干元素合并后的结果的再最终合并的产物。举个例子：

$$[1, 2, 3, 4, 5] \to [(1 + 2 + 3), (4 + 5)] \to 15$$

那么现在设状态 `dp[l][r]` 表示子序列 $a_l, a_{l+1} \cdots, a_r$ 合并成单一元素的最小成本，通过枚举头尾分界位置 m，可以得到转移方程：

$$\texttt{dp[l][r]} = \min\{\texttt{dp[l][m]} + \texttt{dp[m+1][r]} \mid l \le m < r\} + \sum_{i=l}^{r} a_i$$

其中区间和部分可以通过前缀和计算，当然这里由于每个状态有 $O(n)$ 个转移，暴力计算亦可。总复杂度为 $O(n^3)$。

核心代码

```
ll solve(int l, int r) {
    if(l == r) {
        return 0;
    } else {
        if(dp[l][r] != inf) {//已经计算
            return dp[l][r];
        } else {
            for(int m = l; m < r; m++) {
                dp[l][r] = min(dp[l][r], solve(l, m) + solve(m + 1, r));
            }
            for(int i = l; i <= r; i++) {
                dp[l][r] += a[i];
            }
            return dp[l][r];
        }
    }
}
```

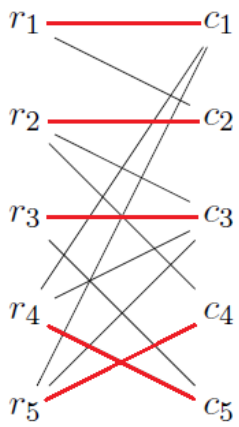
O Matching

题意

给定一个 $n+n$ 个点的二分图，求有多少种完美匹配。

定义：

- 二分图：指顶点可以分成左右两部分，且各自内部顶点间不会有边相连的图。
- 完美匹配：将左右顶点两两配对，但配对的顶点之间必须相邻。



数据范围

- $1 \le n \le 21$

题解

这题是典型的**状态压缩**动态规划，简称状压 DP，一般是指数级的复杂度。尽管复杂度很高，但比最暴力的阶乘级复杂度还是要快不少。

通常我们需要定义一个 `mask` 变量来表示被选取的子集是什么。例如全集是 $\{0,1,2,3,4\}$，而选取的子集是 $\{0,2,3\}$，那么 `mask = 10110`，表达式为 `(1<<0) + (1<<2) + (1<<3)`，其中 `<<` 表示左移。

首先我们考虑暴力做法：依次考虑左边每个点和右边哪个点匹配。假设给定的图是一个二分完全图，即左右之间任意两点均可配对，那么一共有 $n!$ 种匹配方法，在 $n=21$ 的情况下高达 10^{19} 的量级，无法通过。

注意到在考虑顶点 i 的搭档时，我们只关心之前的点已经占用了哪些点，而具体连接方式并不重要。举个例子，假设我们当前考虑顶点 2，之前选择了

$\{(0,3),(1,4)\}$ 还是 $\{(0,4),(1,3)\}$ 对当下的影响是相同的，即不能再使用右边的 3 和 4。

那么我们把等价状态合并：设 dp[i][mask] 表示考虑了节点 $\{0,1,\ldots,i\}$，并且使用的（右侧）点集为 mask。

转移方式很简单：对于状态 dp[i][mask]，只需要枚举节点 i 的邻居 j，如果 mask 包含 j，方案数等于所有 dp[i-1][mask - (1<<j)] 之和。

更进一步地，因为 $\{0,1,\ldots,i\}$ 一共有 $i+1$ 个节点，所以仅当 mask 中 1 的数量等于 $i+1$ 时状态才有意义。于是我们只需要保留 mask 这一维，另一维 i 可以直接设为 __builtin_popcount(mask)-1，其中 __builtin_popcount 表示二进制下 1 的数量。

不难发现，mask 的转移一定是从小的数字转移到大的数字，如 1000 → 1001，所以我们只需要从小到大枚举 mask 即可。

核心代码

```
dp[0] = 1;
for(int mask = 1; mask < (1 << n); mask++) {
    int i = __builtin_popcount(mask) - 1;
    for(int j = 0; j < n; j++) {
        //1. i 和 j 相连
        //2. j 包含在 mask 中
        if(connected[i][j] and (mask & (1 << j)) != 0) {
            dp[mask] += dp[mask - (1 << j)];
        }
    }
}
```

P Independent Set

题意

给一棵 n 个点的树。每个节点可以被染成黑色或白色，但相邻的两个点不能同时染成黑色。求有多少种染色方法。

数据范围

- $1 \le n \le 10^5$

题解

本题是树上的动态规划。套路一般都是由儿子的值转移到父亲，自下而上地动态规划。

这道题也不例外，我们设 dp[cur][0/1] 表示以节点 cur 为根的子树，cur 被染成黑色/白色的方案数。

转移方程非常显然：

把根节点染成黑色时，所有儿子被限制为白色，即：

$$\texttt{dp[cur][0]} = \prod \texttt{dp[son][1]}$$

，其中 son 遍历所有 cur 的儿子。

把根节点染成白色时，儿子没有要求，即：

$$\texttt{dp[cur][1]} = \prod (\texttt{dp[son][0]} + \texttt{dp[son][1]})$$

，其中 son 遍历所有 cur 的儿子。

核心代码

```
void dfs(int cur, int fa = 0) {
    dp[cur][0] = dp[cur][1] = 1;
    for(auto son:edge[cur]) {
        if(son != fa) {
            dfs(son, cur);
            dp[cur][0] *= dp[son][1];
            dp[cur][1] *= dp[son][0] + dp[son][1];
        }
    }
}
dfs(1);
ans = dp[1][0] + dp[1][1];
```

Q Flowers

题意

有 n 朵花，从左到右编号为 1 到 n。每朵花有两个属性：高度 h_i 和美丽值 a_i。

我们需要选出一个子序列（可以不连续），使得：

1. 最大化选择的花美丽值的总和
2. 选择的花的高度从左到右严格递增

数据范围

- $1 \le n \le 2 \times 10^5$
- $1 \le h_i \le n$
- h_i 两两不同
- $1 \le a_i \le 10^9$

题解

朴素做法 我们从前向后考虑，设 `dp[i]` 表示以第 i 朵花结尾的子序列的的最大的美丽值之和。那么一个显然的转移方程是：

$$\texttt{dp[i]} = a_i + \max\{\texttt{dp[j]} \mid j < i \text{ and } h_j < h_i\}$$

可惜复杂度是 $O(n^2)$ 的，但我们可以思考如何优化。

单调性优化 假设我们现在在计算 `dp[i]` 的值，有 j 和 k 两个决策可以考虑。

如果 j 比 k 更优，即 `dp[j]` $>$ `dp[k]` ，并且 $h_j < h_k$，会发生什么呢?

1. 如果可以从 k 转移，因为 $h_j < h_k$，同样也可以从 j 转移。所以不会考虑比 j 劣的 k。
2. 如果无法从 k 转移，那自然不需考虑决策 k。

综上，如果我们只考虑 “有效” 决策，应该尽可能从高度 h 大的决策转移。

现在我们考虑用 `set<pair<long,long>>` 来维护所有有效决策。其中第一维表示高度 h，第二维表示对应的 `dp` 值。在计算 `dp[i]` 时，使用二分找出比 h_i 小的那些决策中最接近的转移即可。

由于题目保证 a_i 均为正数，不需要考虑 `{h[i],dp[i]}` 本身立刻成为无效决策。只需用 `dp[i]` 剔除掉比 h_i 大，但更劣的决策即可。详情参考代码。

核心代码

```
set<pair<long, long>> s;
// 存一个初始决策防止无可用决策
s.insert({0, 0});
for(int i = 1; i <= n; i++) {
    //找出前面高度最接近的
    auto optimal = prev(s.lower_bound({h[i], 0}));
    dp[i] = optimal->second + a[i];
    while(true) {
        auto next = s.lower_bound({h[i], 0});
        // 如果存在之后的决策更劣就剔除出去
        if(next != s.end() and next->second <= dp[i])
            s.erase(next);
        else
            break;
    }
    s.insert({h[i], dp[i]});
}
```

R

题意

题解

核心代码

```
R
```

S

题意

题解

核心代码

T

题意

题解

核心代码

U

题意

题解

核心代码

```
U
```

V

题意

题解

核心代码

```
V
```

W

题意

题解

核心代码

```
W
```

X

题意

题解

核心代码

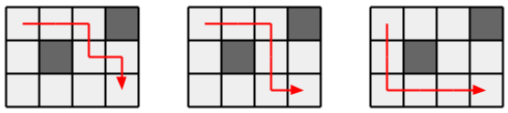
```
X
```

Y　Grid 2

题意

给定一个 h 行 w 列的网格，现在位于 $(1,1)$，每次可以向下或右走一格，求有多少种方式（对 10^9+7 取模）走到 (h,w)。

网格中有 n 个不能经过的障碍，但不会出现在起点和终点。



数据范围

- $2 \le h, w \le 10^5$
- $1 \le n \le 3000$

题解

（注：由于非此题重点，本题解不包含计算组合数取模的教程，请自行上网搜板子）本题是 *Grid 1* 的加强版。由于 $h \times w$ 高达 10^{10}，无法使用之前的方法。接下来我们来使用常见于~~小学奥数~~的**容斥原理**解决这一题。

没有障碍　那么答案就是组合数 $\binom{(h-1)+(w-1)}{h-1}$ ——因为一共要走 $(h-1)+(w-1)$ 步，选择其中的 $h-1$ 步向下。

一个障碍　设障碍格为 (a,b)，那么答案应该是<u>总方案数</u>减去<u>经过障碍格的不合法方案数</u>，即:

$$\underbrace{\binom{(h-1)+(w-1)}{h-1}}_{\text{总方案数}} - \underbrace{\binom{(a-1)+(b-1)}{a-1}\binom{(h-a)+(w-b)}{h-a}}_{\text{经过 (a,b) 的非法方案数}}$$

两个障碍　设障碍格为 $A:(a,b)$ 和 $B:(c,d)$，那么答案等于<u>总方案数</u>减去<u>经过 A 的不合法方案数</u>减去<u>经过 B 的不合法方案数</u>最后加回被计算了两次的<u>同时经过 A 和 B 的不合法方案数</u>。关于最后一部分，这里有两种情况。

1. 不存在同时经过的路径，如上图所示。那么数量就是 0，即答案为

$$
\begin{aligned}
&\binom{(h-1)+(w-1)}{h-1} \\
-&\binom{(a-1)+(b-1)}{a-1}\binom{(h-a)+(w-b)}{h-a} \\
-&\binom{(c-1)+(d-1)}{a-1}\binom{(h-c)+(w-d)}{h-c}
\end{aligned}
$$

2. 存在同时经过的路径。假设先经过 A 然后经过 B，那么方案数为

$$
\begin{aligned}
&\binom{(h-1)+(w-1)}{h-1} \\
-&\binom{(a-1)+(b-1)}{a-1}\binom{(h-a)+(w-b)}{h-a} \\
-&\binom{(c-1)+(d-1)}{a-1}\binom{(h-c)+(w-d)}{h-c} \\
+&\binom{(a-1)+(b-1)}{a-1}\binom{(c-a)+(d-b)}{c-a}\binom{(h-c)+(w-d)}{h-c}
\end{aligned}
$$

多个障碍　总的来说，容斥原理告诉我们，对于障碍的每个子集，如果有奇数个就减去，有偶数个就加上。但 n 个障碍就意味着有 2^n 个子集需要计算，这是无法承受的，我们需要加速。

注意到以下事实：

1. 如果一个子集中存在一个格子在另一个格子的**严格**右上方，那么可以直接忽略，因为我们只能向右下方走，无法同时经过。
2. 设障碍的一个子集**从左上到右下**依次为 $\{(x_1,y_1),(x_2,y_2),\ldots,(x_m,y_m)\}$，那么这个子集的贡献为：

$$
(-1)^m\binom{(x_1-1)+(y_1-1)}{x_1-1}\prod_{i=2}^{m}\binom{(x_i-x_{i-1})+(y_i-y_{i-1})}{x_i-x_{i-1}}\binom{(h-x_m)+(w-y_m)}{h-x_m}
$$

3. 如果把起点和终点也看作的障碍格，那么形式更加简洁：

$$
(-1)^m\prod_{i=2}^{m}\binom{(x_i-x_{i-1})+(y_i-y_{i-1})}{x_i-x_{i-1}}
$$

经过以上铺垫，我们大致可以窥到完整解法的轮廓了。首先把所有障碍（以及起点和终点）按左上到右下的顺序排序。那么我们设 `dp[i][0/1]` 表示走到第 i

个格子一共经过了奇数/偶数个障碍。那么转移方程为：

$$\texttt{dp[i][0/1]} = \sum_{\text{j 在 i 的左上方}} \texttt{dp[j][1/0]} \binom{(x_i - x_j) + (y_i - y_j)}{x_i - x_j}$$

初始条件为：

$$\texttt{dp[(1,1)][1]} = 1$$

答案为：

$$\texttt{dp[(h,w)][0]} - \texttt{dp[(h,w)][1]}$$

总复杂度是 $O(n^2)$。

核心代码

```
dp[0][1] = 1;
for(int i=1;i<n+2;i++) {
    for(int j = 0;j<i;j++){
        if(x[j]<=x[i] and y[j]<=y[i]){
            for(int parity = 0;parity<2;parity++){
                dp[i][0] += dp[j][1] * binomial(x[i]-x[j]+y[i]-y[j],x[i]-x[j])
            }
        }
    }
}
ans = dp[n-1][0] - dp[n-1][1]
```

Z　Frog 3

题意

有 n 块石头（编号为 1 到 n）排成一排，第 i 块石头的高度为 h_i，并满足 $h_1 < h_2 < \cdots < h_N$。青蛙现在站在第 1 块石头上。它可以从第 i 块石头跳到第 $i+1, i+2, \ldots, N$ 块石头上。从第 i 块石头跳到第 j 块需要消耗 $(h_j - h_i)^2 + C$ 点体力。请求出跳到第 n 块石头所需的最小体力值。

数据范围

- $2 \le N \le 2 \times 10^5$
- $1 \le C \le 10^{12}$
- $1 \le h_1 < h_2 < \cdots < h_N \le 10^6$

题解

这题用到的技巧被称为**决策单调性**。这是一个貌似高端，但本质简单的知识点。下面我来分别介绍一下什么是**决策**和**单调性**。

什么是决策　首先，我们依旧设 `dp[i]` 为跳到第 i 块石头上所需的最小体力值。其次，用 $\texttt{cost}(i,k)$ 表示从 i 转移到 k 得到的值，表达式为 $\texttt{dp[i]}+(h_k-h_i)^2+C$。显然有 $\texttt{dp[k]} = \min\{\texttt{cost}(i,k) \mid 1 \le i < k\}$。在计算 `dp[k]` 的时候，每一个转移源头 i 被称为一个**决策**。

枚举所有的决策的总复杂度是 $O(n^2)$。这显然是不可承受的，于是我们需要探索更多性质。

什么是单调性　假设在考虑 `dp[k]` 的时候，决策 j 要优于决策 i，即 $\texttt{cost}(i,k) > \texttt{cost}(j,k)$，我们看看可以为计算 `dp[k+1]` 提供什么便利。根据 $\texttt{cost}(i,k)$ 的定义，我们可以得到决策 i 的成本增幅为：

$$
\begin{aligned}
\Delta_i &= \texttt{cost}(i,k+1) - \texttt{cost}(i,k) \\
&= \underbrace{(\texttt{dp[i]} + (h_{k+1}-h_i)^2 + C)}_{\texttt{cost}(i,k+1)} - \underbrace{(\texttt{dp[i]} + (h_k-h_i)^2 + C)}_{\texttt{cost}(i,k)} \\
&= (h_{k+1}-h_i)^2 - (h_k-h_i)^2 \\
&= (h_{k+1}+h_k-2h_i)(h_{k+1}-h_k) \ \textbf{(平方差公式)}
\end{aligned}
$$

同理可得 $\Delta_j = (h_{k+1}+h_k-2h_j)(h_{k+1}-h_k)$

注意到 $h_i < h_j < h_k < h_{k+1}$ 是**单调递增**的，不难计算 $\Delta_i - \Delta_j = (2h_j - 2h_i)(h_{k+1}-h_k) > 0$，即决策 i 的成本上升得比决策 j 更多。由于决策 j 本来

就是更优的决策，它将一直保持优势下去。于是我们可以得出结论：**一旦一个决策被后来者打败，那么再也不需要被考虑**。

在计算一个状态的值的时候，我们定义成本最小的决策为最优决策，如有多种取最左的。举个例子：

i	1	2	3	4	5
最优决策	N/A	1	1	2	3

由于之前的结论，最优决策从左到右一定是单调不下降的：只有可能出现 111222333，而不会有 111333222。因为一旦 2 被 3 打败，就永远不会比 3 更优了。这就是所谓的**单调性**。

由于决策具有单调性，我们只需要二分出每个决策作为最优决策的起点即可。我们这里用pair<int,int>的数组实现，每个元素(i,pos) 表示决策 i 从 pos。开始成为最优策略。例如我们用 [(1,1),(2,4),(3,7)] 表示 111222333。

鉴于文字描述会过于冗长，直接观察以下伪代码：

核心代码

```
dp = [0,inf,inf,...] # 下标从 0 开始
optimal_strategy = [(1,2)] # 一开始 0 是所有状态的最优策略
for j in [2..n]:
# 首先二分出当前位置的最优策略
    l = 1, r = optimal_strategy.length()
    while l < r:
        mid = (l + r) / 2 + 1
        if optimal_strategy[mid][0] <= j:
            l = mid
        else:
            r = mid-1
    dp[j] = cost(optimal_strategy[l][0],j)
# 然后我们计算成为最优决策的起点
    # 首先去除完全劣于当前策略的策略
    i,pos = optimal_strategy.back()
    while cost(i,pos) > cost(j,pos):
        optimal_strategy.pop()
        i,pos = optimal_strategy.back()
    # 开始二分起点
    l = pos, r = n
    while l < r:
        mid = (l + r) / 2
        if cost(i,mid) <= cost(j,mid):
            l = mid + 1
        else:
            r = mid
    optimal_strategy.push((j,l)); # 二分后 l 就是想求的起点
```